

SPRINT 01

Technical Lead Architecture Decision Package

Industrial Equipment Monitoring Platform

Document	Technical Lead Architecture Decision Package
Milestone	M1 - Contracts and Vertical Slice
Role	Technical Lead
Requirements baseline	Requirements Specification v1.0 - Draft baseline
Release	Portfolio MVP
Status	Draft for architecture review

Purpose: define component boundaries and deployment topology, and document the transport, durable-buffering, and duplicate-protection decisions required for the MVP reliability architecture.

1. Architecture Review Scope

This package captures the Technical Lead architecture baseline for Sprint 01. It defines the runtime boundaries and communication contracts that later gateway reliability work can implement without changing the telemetry contract.

Technical Lead position: Sprint 01 remains contract-first. Full persistent gateway buffering, retry, restart recovery, and the wider reliability campaign are primarily implemented in M3, but their interfaces and semantics are frozen here so Firmware, Gateway, Backend and QA can work independently.

Acceptance criteria

- Architecture reviewed.
- Three ADRs committed with rationale and trade-offs.
- Open decisions explicitly marked.
- Backend can be interrupted independently of local MQTT.

Source basis

This document converts the previously agreed Technical Lead architecture review into a formal project artifact. It is grounded in the project Requirements Specification v1.0, the Sprint 01 QA telemetry decision log, and the Sprint 01 Technical Report. It does not change the requirements baseline.

2. Component Boundaries

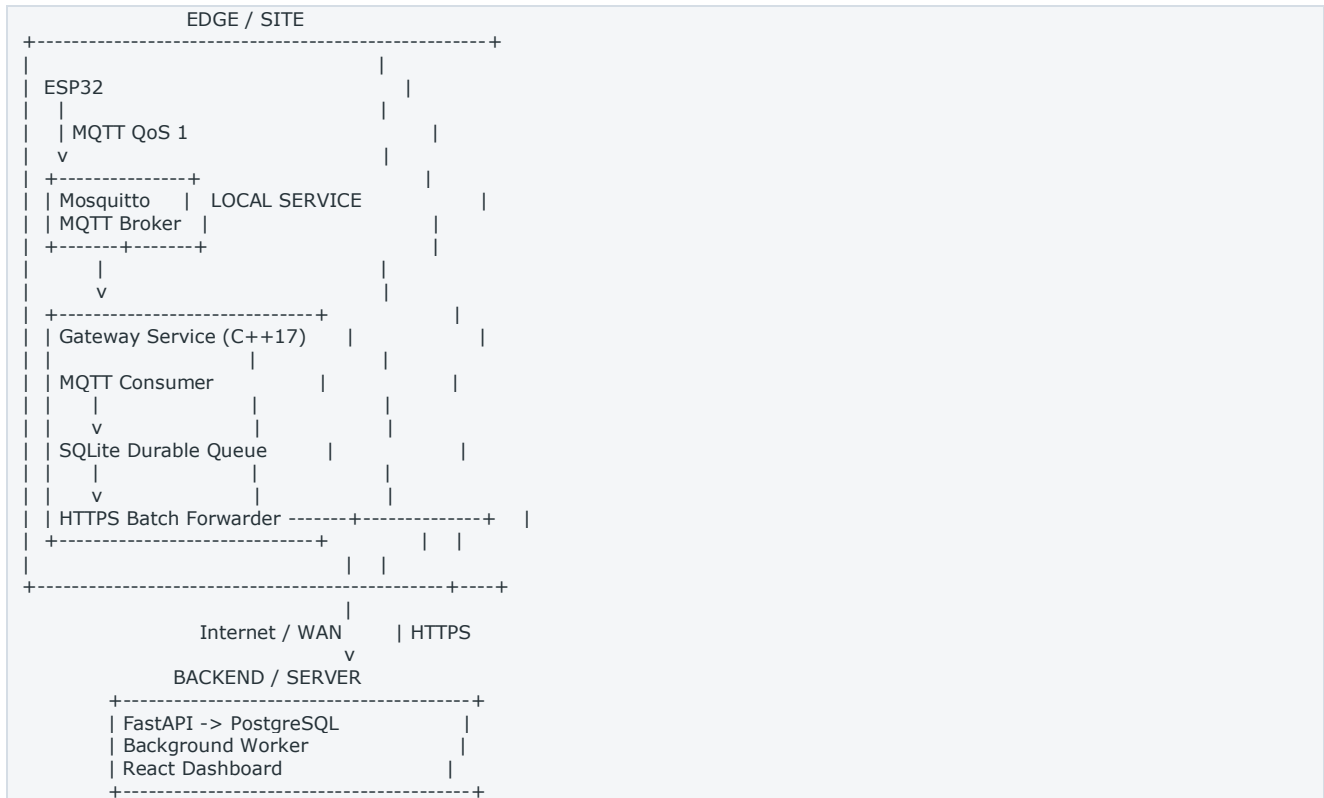
The architecture separates local acquisition and durable edge handling from remote backend processing. This allows the local MQTT path to remain operational even when the backend or WAN is unavailable.

Component	Runtime / Technology	Owns	Must not own
ESP32 firmware	ESP32 / ESP-IDF / C/C++	Sensor acquisition; boot_id; sequence_number; measured_at; uptime; MQTT publish	Durable backend delivery; PostgreSQL knowledge
Local MQTT broker	Raspberry Pi / Mosquitto	Local device-to-gateway message transport	Durable application telemetry queue; backend acknowledgement
Gateway MQTT consumer	Raspberry Pi / C++17	MQTT subscription; basic schema checks; gateway_received_at	Backend business rules
Gateway durable queue	Raspberry Pi / SQLite	Persistent unsent events; retry state; quarantine	Global telemetry history
Gateway forwarding worker	Raspberry Pi / C++17	HTTPS batching; retry/backoff; response processing	Determining backend persistence independently
Backend ingestion API	Server / FastAPI	Authentication; ownership validation; idempotency; permanent validation; transaction boundary	Gateway retry scheduling
PostgreSQL	Server	Authoritative telemetry storage; uniqueness constraint	Edge buffering
Background worker	Server / Python	Retention; connection-state jobs	Local MQTT
Dashboard	Browser / React + TypeScript	Visualization and operator workflow	Telemetry authority

Responsibility boundary: Backend decides the event outcome. Gateway decides the queue lifecycle from that outcome.

3. Deployment Topology

The deployment topology deliberately separates the edge/site runtime from the backend/server runtime. Local MQTT and gateway persistence remain available during a backend outage.



The critical failure boundary is:

```

ESP32 -> Mosquitto -> Gateway -> SQLite
      |
      X Backend unavailable

```

A backend outage therefore affects the forwarding path, not local telemetry transport or the durable gateway queue.

4. ADR-003 - Gateway-to-Backend Transport

Status: Proposed for acceptance

Decision

Use HTTPS request/response batch ingestion between the Raspberry Pi gateway and backend. Continue using MQTT QoS 1 only for ESP32 -> local Mosquitto -> gateway transport.

Device domain:
ESP32 --MQTT QoS 1--> Mosquitto --MQTT--> Gateway

Server domain:
Gateway --HTTPS batch--> Backend

Context and rationale

The device and server communication domains have different reliability responsibilities. MQTT QoS 1 provides at-least-once delivery locally, so duplicate messages remain possible. A broker acknowledgement also does not prove that the gateway has durably committed an event to SQLite. HTTPS gives the gateway an application-level request/response contract with deterministic per-event outcomes.

accepted
duplicate
rejected

This maps cleanly to queue actions: remove an accepted event, remove a matching duplicate, quarantine a permanent rejection, and retain transient failures for retry.

Trade-offs

Benefit	Cost / trade-off
Simple FastAPI integration	Gateway requires retry logic
Natural request/response semantics	Not a streaming protocol
Clear gateway authentication boundary	Large backlog requires batching
Database commit can be acknowledged explicitly	HTTP connection and serialization overhead
Easy to exercise lost-response scenario AT-05	Gateway must classify HTTP/network failures
Backend can evolve independently of MQTT	Two transport protocols exist

Rejected alternatives

MQTT directly into the backend. Rejected for the MVP because it would couple local broker semantics to server availability and blur the edge/backend reliability boundary.

Kafka or RabbitMQ between gateway and backend. Rejected for the MVP because it adds infrastructure beyond the current scale and acceptance needs.

Open decisions

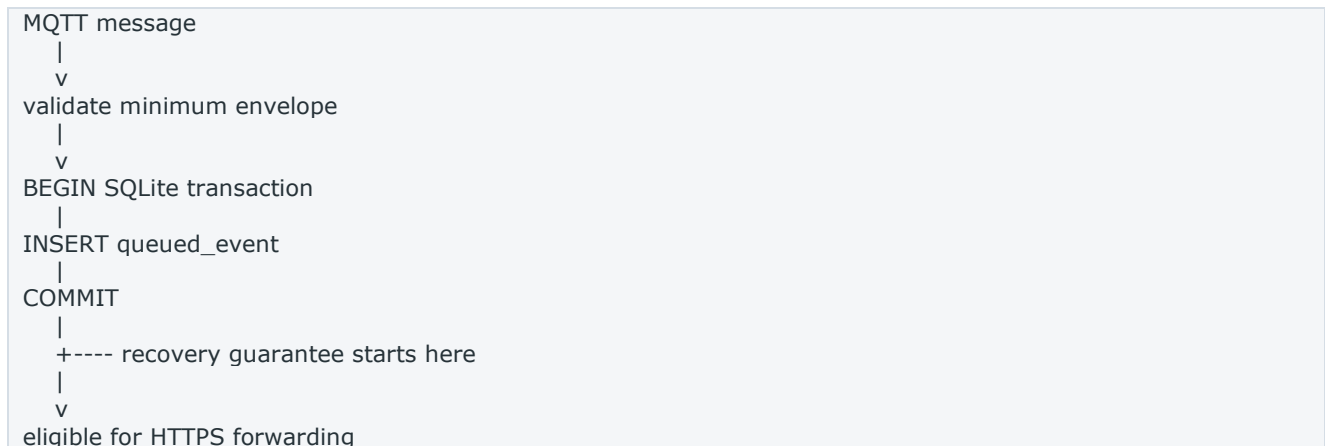
- OD-TRANSPORT-01 - Final HTTPS timeout values.
- OD-TRANSPORT-02 - Exact bounded exponential-backoff parameters and jitter algorithm.
- OD-TRANSPORT-03 - Exact HTTP status semantics for a syntactically valid mixed-result batch.

5. ADR-004 - Durable Gateway Buffering

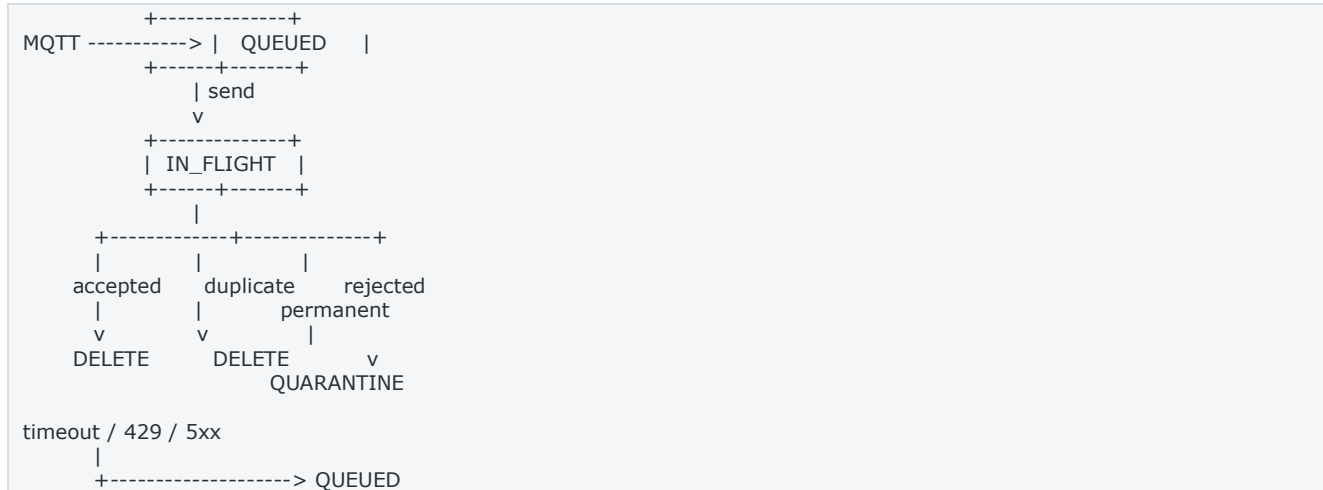
Status: Proposed for acceptance

Decision

Use SQLite as the durable application queue on the Raspberry Pi. The gateway must commit telemetry to SQLite before treating that telemetry as recoverable.



Queue state model



Rationale

SQLite provides transactional durability without introducing another edge service. It supports restart recovery, bounded queue management, inspection, metrics, and quarantine while remaining appropriate for a single Raspberry Pi gateway process.

Trade-offs

Benefit	Cost / trade-off
Survives gateway-process restart	Requires queue schema and migrations
No separate queue server	Database locking/concurrency must be handled
Transactional durability	Flash/storage wear must be considered
Easy queue-depth and oldest-event metrics	Corruption/recovery procedure is required
Supports bounded quarantine	Queue cleanup/retention logic is required
Works while WAN/backend is offline	Capacity must be bounded and exhaustion visible

Open decisions

- OD-BUFFER-01 - Maximum configured queue capacity beyond the minimum NFR target.
- OD-BUFFER-02 - Quarantine maximum size and retention period.
- OD-BUFFER-03 - SQLite journaling and synchronous settings, such as WAL and durability level.
- OD-BUFFER-04 - Whether IN_FLIGHT is persisted explicitly or derived from retry metadata.

6. ADR-005 - Duplicate Protection and Idempotent Ingestion

Status: Proposed for acceptance

Decision

Define telemetry event identity as the composite key (device_id, boot_id, sequence_number) and enforce this identity in PostgreSQL with a unique constraint.

```
event_identity = (device_id, boot_id, sequence_number)
```

```
UNIQUE (device_id, boot_id, sequence_number)
```

Duplicate rules

```

Identity absent
-> INSERT
-> accepted

Identity exists + immutable content identical
-> duplicate
-> do not insert

Identity exists + immutable content different
-> rejected_identity_conflict
-> log conflict

```

For duplicate comparison, receipt timestamps are not immutable device content because gateway/backend receipt times naturally differ across retries. Immutable content should include:

- schema_version
- device_id
- boot_id
- sequence_number
- measured_at
- device_uptime_ms
- metric
- value
- unit
- quality

Critical lost-response scenario

```

Gateway
| POST event #100
v
Backend
| INSERT
| COMMIT
X response lost
|
Gateway timeout
| retry same identity
v
Backend
| UNIQUE identity already exists
| compare immutable content
v
duplicate

Database rows = 1

```

Reliability statement: At-least-once delivery + idempotent processing -> one authoritative stored telemetry record. Do not claim exactly-once transport.

Transaction boundary

An accepted result may be returned only after the PostgreSQL transaction commits. If the gateway cannot know whether a commit succeeded because the response was interrupted, it retries the same event identity and duplicate protection keeps the stored row count correct.

Trade-offs

Benefit	Cost / trade-off
Deterministic retries	Additional conflict/lookup logic
Protects against lost backend responses	Immutable-content definition must be stable
Handles MQTT QoS 1 duplicates	Database unique index is required
Simple identity generation	boot_id generation must be reliable
One authoritative stored row	Transport itself remains at-least-once

Open decisions

- OD-DUP-01 - Compare immutable database columns directly versus storing a canonical content hash.
- OD-DUP-02 - Canonical normalization rules before comparison, particularly floating-point representation.
- OD-DUP-03 - Concurrency handling if two requests carrying the same identity arrive simultaneously.

7. Backend-Independent Local MQTT Verification

The acceptance condition "backend can be interrupted independently of local MQTT" should be verified as an explicit architecture test. The local acquisition path must continue while the remote backend is stopped or unreachable.

```

T0 ESP32 publishing normally
Mosquitto running
Gateway running
Backend running
queue_depth = 0

T1 Stop FastAPI or block the backend route.

T2 ESP32 continues publishing.
Mosquitto remains running.
Gateway MQTT subscriber remains connected.
SQLite queue_depth increases.

T3 Keep backend unavailable.

T4 Restart gateway.
Mosquitto stays available.
Persisted queue survives.

T5 Restore backend.

T6 Gateway retries backlog.
queue_depth -> 0.

T7 Verify PostgreSQL.
All committed events exist.
Duplicate identities = 0.

```

Expected evidence

- Mosquitto process/service remains healthy during backend interruption.
- Gateway remains subscribed to the local MQTT topics.
- SQLite queue depth and oldest queued-event age increase during the outage.
- Gateway restart does not delete previously committed queue rows.
- After backend recovery, the backlog drains while new readings continue.
- PostgreSQL contains one authoritative row per event identity.
- No manual gateway restart is required after backend recovery.

8. Architecture Review Status

Acceptance criterion	Technical Lead result
Component boundaries defined	Ready for review
Deployment topology defined	Ready for review
ADR: transport	Draft complete - ADR-003
ADR: durable buffering	Draft complete - ADR-004
ADR: duplicate protection	Draft complete - ADR-005
Rationale documented	Yes
Trade-offs documented	Yes
Open decisions explicitly marked	Yes
Backend interrupt independent from local MQTT	Architecturally satisfied; implementation test pending
Three ADRs committed to repository	Pending repository commit
Architecture reviewed by team	Pending team review

9. Recommended Repository Structure

The new ADRs should sit beside the identity and timestamp decisions already established in the Sprint 01 Technical Report:

```
docs/
|-- architecture/
|   |-- architecture-overview.md
|   |-- deployment-topology.md
|
|-- adr/
|   |-- ADR-001-telemetry-event-identity.md
|   |-- ADR-002-timestamp-ownership.md
|   |-- ADR-003-gateway-backend-transport.md
|   |-- ADR-004-durable-gateway-buffering.md
|   |-- ADR-005-duplicate-protection.md
```

10. Technical Lead Completion Statement

ADR-003 through ADR-005 are the three architecture records required by the current Technical Lead task. ADR-001 and ADR-002 preserve the event-identity and timestamp-ownership decisions already recorded for Sprint 01. Once the three new ADRs are reviewed and committed, the remaining acceptance evidence is the executable backend-interruption test demonstrating that local MQTT and gateway persistence continue independently of the remote backend.

References

- Industrial Equipment Monitoring Platform - Requirements Specification, Version 1.0 Draft baseline. Relevant sections: Proposed implementation baseline; FR-05 to FR-09; telemetry processing rules; NFR-04 to NFR-07; AT-03 to AT-06; Definition of Done.
- QA Sprint 1 - Telemetry Contract, Decision Log and Open Questions. Relevant sections: canonical event identity; partial batch outcomes; lost response after commit; open batch questions; QA decision register.
- Sprint 01 Technical Report - Technical Lead Report, M1 Contracts and Vertical Slice. Relevant sections: Event Identity and Idempotency; Gateway Queue Lifecycle; Technical Decision Register; Open Questions.